

Rapport d'activité – Rendu 2

Part I : Méthode de travail

Nous avons conservé la même méthode de travail que lors du Rendu 1. Celle-ci repose sur une phase initiale de planification commune, suivie d'un développement autonome des différentes parties du projet, puis d'une intégration finale réalisée ensemble.

Nous avons commencé par prendre en main les fichiers GTKmm fournis par l'EPFL, afin de comprendre l'architecture du projet, avant de répartir les tâches entre les membres du groupe.

Part II : Répartition du travail

Liam Van Camp

Mise à jour de la classe `Paddle` (`target_mouse`, `move_to`, `is_valid` avec `eps`), mise à jour du module `game` (`save`, `reset`, `update`, getters, `move_paddle_to`), ainsi que la structure de base du module `gui` (boutons, boucle de jeu).

Victor Henri Willy Eder

Implémentation des raccourcis clavier dans le module `gui` (touches `s`, `1`, `r` reliées aux boutons `start`, `step`, `restart`), mise à jour des classes `Ball` et `Brick` (surcharge de `is_valid` avec `eps`), complétion du module `gui` (interactions souris), tests et validation de l'interface graphique.

Travail effectué ensemble

Développement du module `graphic` (dessin Cairo des briques, balles, raquette et arène), intégration finale, mise en place du Makefile GTKmm, corrections et validation sur les fichiers de test.

Part III : Hiérarchie des classes Brick

La superclasse `Brick` contient les attributs `body` (de type `Square`) et `type` (de type `BrickType`), ainsi que les méthodes `is_valid()`, `is_inside_arena()` et `is_size_valid()`. La méthode `is_valid()` est déclarée `virtual` afin de permettre un comportement adapté dans les classes dérivées, en appelant la version correspondant à l'objet réel (polymorphisme). Le destructeur est également `virtual` afin d'assurer une libération correcte de la mémoire lors de l'utilisation de pointeurs de type `Brick*`.

La classe `RainbowBrick` ajoute l'attribut `hit_points` et redéfinit (`override`) la méthode `is_valid()` afin d'y inclure la vérification `is_hit_points_valid()`.

Les classes `BallBrick` et `SplitBrick` n'ajoutent actuellement ni attributs ni méthodes spécifiques, mais permettent d'étendre leur comportement ultérieurement.

Part IV : Structure des données de Game

La classe `Game` regroupe l'état global de la partie et coordonne les autres entités. Ses attributs principaux sont `score` et `lives`, qui mémorisent l'état du joueur, ainsi qu'un objet `Paddle` `paddle`, un ensemble de balles `std::vector<Ball> balls` et un ensemble de briques `std::vector<Brick*> bricks`.

Les différentes entités du jeu sont donc mémorisées dans des conteneurs dynamiques au sein de `Game`. La raquette est stockée comme un objet unique, car il n'existe qu'une seule instance en jeu. Les balles sont stockées par valeur dans un vecteur homogène, tandis que les briques sont stockées sous forme de pointeurs vers la classe de base `Brick`, afin de permettre le polymorphisme entre les différentes classes dérivées.

Les dépendances entre modules sont les suivantes :

```
tools ← brick/ball/paddle ← game ← gui ← project
tools ← graphic ← gui
```

Part V : Module tools

Le module `tools` définit les structures géométriques de base : `Point` (vecteur 2D), `Circle` (centre et rayon) et `Square` (centre et demi-longueur de côté).

Il fournit également des fonctions utilitaires telles que `norm()`, `distance()`, `normalized()`, `contains()` et `intersects()` (pour les combinaisons `Circle/Circle`, `Square/Square` et `Circle/Square`).

Ce module est indépendant de toute logique de jeu.